

For Each In 2D (Java)



Purpose and Context

The program `ForEachIn2D.java` was intended to demonstrate how two-dimensional arrays in Java could be created, initialized, and traversed. It worked specifically on a 3 x 2 integer array:

```
int[][] nums = new int[3][2];
```

This structure gave the program three rows and two columns, which provided a small but sufficient dataset to show both **indexed iteration** and **enhanced for-each iteration**. The overall goal was not only to assign values but also to process them, which concluded with the display of individual elements and their summation.



Control Flow and Structure

The program began by printing the length of rows with:

```
System.out.println("Length of rows: " + nums.length);
```

Since `nums` was declared as `new int[3][2]`, the row length evaluated to 3.

To check the number of columns, the program accessed the first row with `nums[0].length`, but only after ensuring that the array contained rows:

```
if (nums.length > 0) {  
    System.out.println("Length of columns: " + nums[0].length);  
}
```

This condition acted as a safeguard. Without it, attempting to access `nums[0].length` in an empty array would have caused a runtime error.

For initialization, nested for loops were used:

```
for (int row = 0; row < nums.length; row++) {  
    for (int col = 0; col < nums[row].length; col++) {  
        nums[row][col] = (row + 1) * (col + 1);  
    }  
}
```

Here, `(row + 1) * (col + 1)` defined the formula for filling the array. It produced a multiplication table effect, which created values like 1, 2 in the first row, and 2, 4 in the second. This choice of

formula highlighted the relationship between row and column indices.

After filling the array, a **for-each loop** was applied:

```
for (int[] rvals : nums) {  
    for (int cvals : rvals) {  
        System.out.print(cvals + " ");  
        sum += cvals;  
    }  
    System.out.println();  
}
```

This structure navigated each row (`int[] rvals`) as a one-dimensional array, and then iterated over each value (`cvals`). Unlike the traditional loop, it did not require index management. This made the traversal simpler, which emphasized the readability advantage of the for-each loop once data had already been populated.



Behavior and Output

The nested indexed loops assigned values systematically. The trace of assignments was as follows:

Step	Row & Column	Action	Running Sum
Init 1	(0,0)	Assign <code>nums[0][0] = 1</code>	–
Init 2	(0,1)	Assign <code>nums[0][1] = 2</code>	–
Init 3	(1,0)	Assign <code>nums[1][0] = 2</code>	–
Init 4	(1,1)	Assign <code>nums[1][1] = 4</code>	–
Init 5	(2,0)	Assign <code>nums[2][0] = 3</code>	–
Init 6	(2,1)	Assign <code>nums[2][1] = 6</code>	–
Trav 1	(0,0)	Read 1, add to sum	1
Trav 2	(0,1)	Read 2, add to sum	3
Trav 3	(1,0)	Read 2, add to sum	5
Trav 4	(1,1)	Read 4, add to sum	9
Trav 5	(2,0)	Read 3, add to sum	12

Trav 6	(2,1)	Read 6, add to sum	18
--------	-------	--------------------	----

This led to the following 2D structure:

```
1 2
2 4
3 6
```

The summation of all elements was handled inside the for-each loop by:

```
sum += cvals;
```

which accumulated the values into sum. At the end, the program printed:

```
Summation: 18
```

Below is the sample console output that reflects the program's behavior:

```
Length of rows: 3
Length of columns: 2
1 2
2 4
3 6
Summation: 18
```